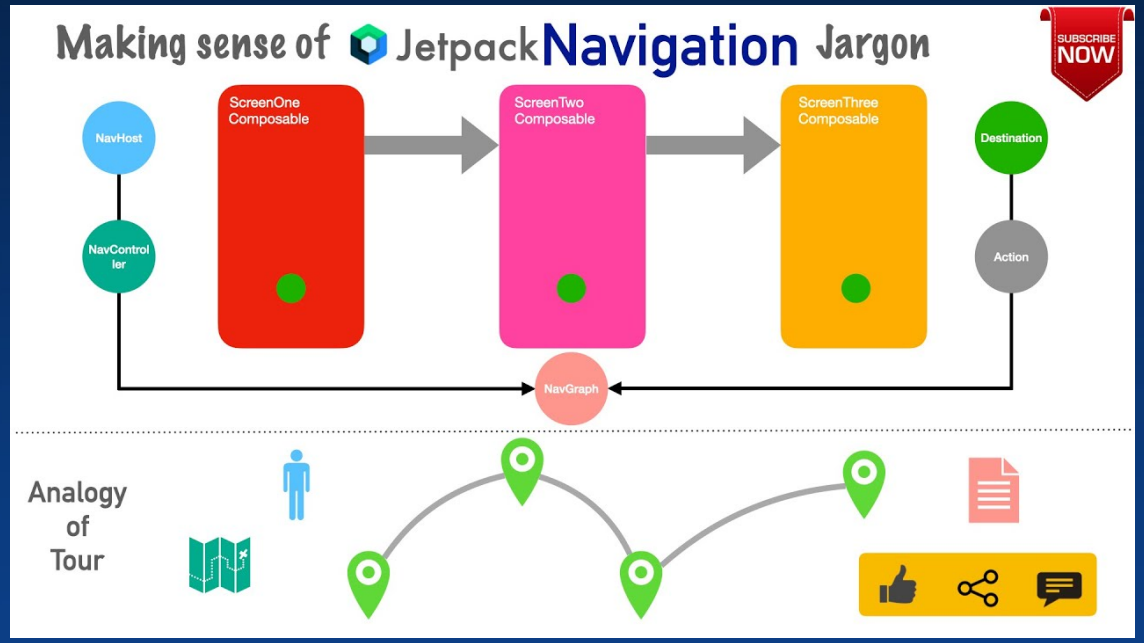
UD3. Menús de navegación

0. índice

1. Tipos de navegación
2. Ejemplo de menú de navegación con NavController
3. NavControler y ViewModel

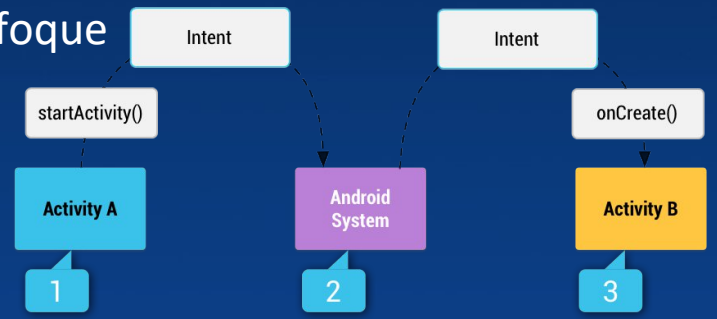
1. Tipos de navegación

- Existen dos enfoques para menús de navegación:
  - Navegación entre actividades usando intents
  - Navegación entre composables usando NavController



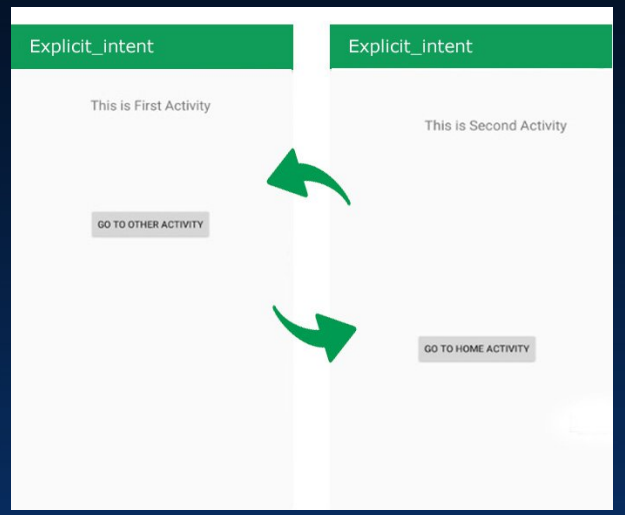
1. Tipos de navegación

- Navegación entre actividades usando intents: Este enfoque implica navegar entre diferentes actividades de Android utilizando Intents. Cada pantalla de la aplicación se representa como una actividad independiente.
  - Ventajas:
    - Separación clara: Cada actividad tiene su propio ciclo de vida, lo que es útil si las pantallas son muy diferentes y no necesitan compartir estados o datos.
    - Facilidad inicial: Para aplicaciones simples con pocas pantallas, este enfoque puede ser más sencillo de implementar.



1. Tipos de navegación

- Navegación entre actividades usando intents:
  - Desventajas:
    - Manejo de estado complejo: Compartir datos entre actividades es más complicado y requiere pasar datos con Intent extras o utilizar ViewModel compartidos.
    - Mayor sobrecarga: Cada actividad debe ser creada y gestionada por el sistema, lo que implica más consumo de recursos y puede resultar en transiciones menos fluidas.
    - Menor flexibilidad: Las transiciones y animaciones son más lmitadas cuando se navega entre actividades.



1. Tipos de navegación

Código del `BottomNavBar`:

```
@Composable
fun BottomNavBar(navController: NavHostController) {
    val context = LocalContext.current
    val coroutineScope = rememberCoroutineScope()
    val currentActivity = context.javaClass.simpleName

    NavigationBar {
        NavigationBarItem(
            icon = { Icon(Icons.Default.Home, contentDescription = "Home") },
            selected = currentActivity == "MainActivity", // Verifica si es la actividad actual
            onClick = {
                coroutineScope.launch {
                    context.startActivity(Intent(context, MainActivity::class.java))
                }
            }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Default.Info, contentDescription = "Info") },
            selected = currentActivity == "InfoActivity",
            onClick = {
                coroutineScope.launch {
                    context.startActivity(Intent(context, InfoActivity::class.java))
                }
            }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Default.Brush, contentDescription = "Gallery") },
            selected = currentActivity == "GalleryActivity",
            onClick = {
                coroutineScope.launch {
                    context.startActivity(Intent(context, GalleryActivity::class.java))
                }
            }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Default.Settings, contentDescription = "Settings") },
            selected = currentActivity == "SettingsActivity",
            onClick = {
                coroutineScope.launch {
                    context.startActivity(Intent(context, SettingsActivity::class.java))
                }
            }
        )
    }
}
```

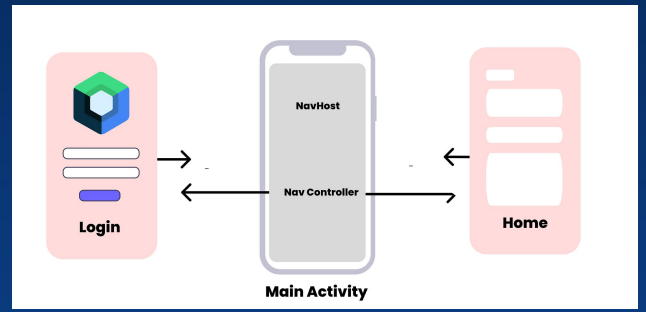
Definición de las actividades en `AndroidManifest.xml`:

```
<activity android:name=".MainActivity" />
<activity android:name=".InfoActivity" />
<activity android:name=".GalleryActivity" />
<activity android:name=".SettingsActivity" />
```

- Navegación entre actividades usando intents:
  - En este enfoque, cada clic de un ítem del menú lanza una nueva actividad con un Intent, y la actividad actual se destaca en el menú con selected = currentActivity == “NombreDeLaActividad”

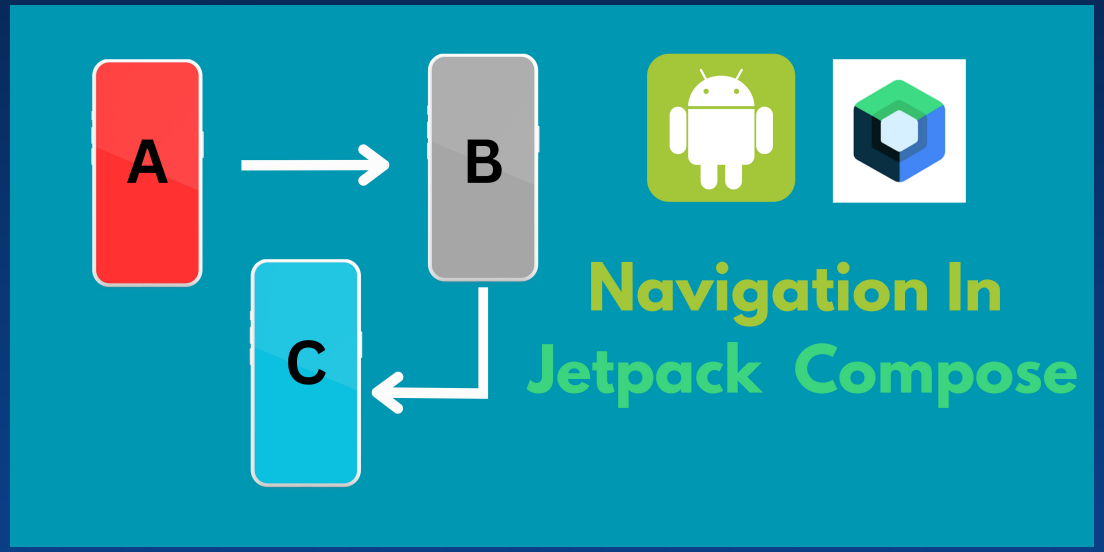
1. Tipos de navegación

- Navegación entre composables usando NavController: Este enfoque utiliza NavController de Jetpack Compose para navegar entre diferentes composables dentro de una sola actividad. Todas las pantallas están en la misma actividad y se gestionan a través de un sistema de navegación interno.
  - Ventajas:
    - Mejor manejo del estado: Al estar todas las pantallas en la misma actividad, puedes compartir datos fácilmente entre composables utilizando un ViewModel compartido.
    - Transiciones más fluidas: Navegar entre composables es mucho más rápido y eficiente que cambiar entre actividades, lo que resulta en una experiencia de usuario más suave.
    - Mayor flexibilidad: Puedes agregar transiciones animadas más fácilmente y gestionar la navegación de una manera más escalable.



1. Tipos de navegación

- Navegación entre composables usando NavController:
  - Desventajas:
    - Curva de aprendizaje: Si no estás familiarizado con NAvController, el sistema de navegación de Jetpack Compose, puede costar un poco al principio.



1. Tipos de navegación

- Navegación entre composables usando NavController:

Código del `BottomNavBar` con `NavController`:

```
@Composable
fun BottomNavBar(navController: NavHostController) {
    NavigationBar {
        NavigationBarItem(
            icon = { Icon(Icons.Default.Home, contentDescription = "Home") },
            selected = navController.currentBackStackEntry?.destination?.route == "home",
            onClick = {
                navController.navigate("home") {
                    popUpTo(navController.graph.startDestinationId) { saveState = true }
                    launchSingleTop = true
                    restoreState = true
                }
            }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Default.Info, contentDescription = "Info") },
            selected = navController.currentBackStackEntry?.destination?.route == "info",
            onClick = {
                navController.navigate("info") {
                    popUpTo(navController.graph.startDestinationId) { saveState = true }
                    launchSingleTop = true
                    restoreState = true
                }
            }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Default.Brush, contentDescription = "Gallery") },
            selected = navController.currentBackStackEntry?.destination?.route == "gallery",
            onClick = {
                navController.navigate("gallery") {
                    popUpTo(navController.graph.startDestinationId) { saveState = true }
                    launchSingleTop = true
                    restoreState = true
                }
            }
        )
        NavigationBarItem(
            icon = { Icon(Icons.Default.Settings, contentDescription = "Settings") },
            selected = navController.currentBackStackEntry?.destination?.route == "settings",
            onClick = {
                navController.navigate("settings") {
                    popUpTo(navController.graph.startDestinationId) { saveState = true }
                    launchSingleTop = true
                    restoreState = true
                }
            }
        )
    }
}
```

Configuración del `NavHost` en la actividad principal:

```
@Composable
fun MainScreen() {
    val navController = rememberNavController()

    Scaffold(
        bottomBar = { BottomNavBar(navController) }
    ) {
        NavHost(navController = navController, startDestination = "home") {
            composable("home") { HomeScreen(navController) }
            composable("info") { InfoScreen(navController) }
            composable("gallery") { GalleryScreen(navController) }
            composable("settings") { SettingsScreen(navController) }
        }
    }
}
```

Definir cada pantalla como un composable:

```
@Composable
fun HomeScreen(navController: NavController) {
    // Contenido de la pantalla Home
}

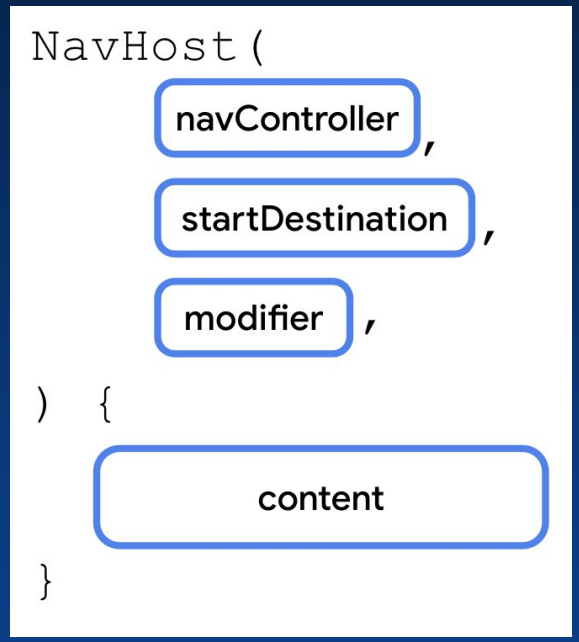
@Composable
fun InfoScreen(navController: NavController) {
    // Contenido de la pantalla Info
}

@Composable
fun GalleryScreen(navController: NavController) {
    // Contenido de la pantalla Gallery
}

@Composable
fun SettingsScreen(navController: NavController) {
    // Contenido de la pantalla Settings
}
```

1. Tipos de navegación

- Conclusión:
  - Navegación entre actividades con Intents:
    - Útil para aplicaciones pequeñas y sencillas, donde cada pantalla es completamente independiente.
    - Puede ser ineficiente si las pantallas necesitan compartir datos o si hay muchas transiciones entre ellas.
  - Navegación entre composables con NavController:
    - Recomendado para aplicaciones que escalan en complejidad, donde las pantallas comparten estado o datos.
    - Ofrece una experiencia de usuario más fluida, mejor gestión de estado y animaciones entre pantallas.
  - En resumen, si tu aplicación tiene múltiples pantallas relacionadas o compartes datos entre ellas, usar NavController es la opción más eficiente y flexible. Si tienes una aplicación pequeña con pantallas independientes, usar Intents es más simple.



2. Ejemplo de menú de navegación con NavController

```
class MainActivity : ComponentActivity() {
        override fun onCreate(savedInstanceState: Bundle?) {
            super.onCreate(savedInstanceState)
            enableEdgeToEdge()
            setContent {
                ViewContainer()
                }
            }
        }


    @Preview(showBackground = true)
    @SuppressLint("UnusedMaterial3ScaffoldPaddingParameter")
    @Composable
    fun ViewContainer() {
        val navController = rememberNavController()
        MaterialTheme {
            Scaffold(
                content = {
                    NavHost(
                        navController = navController,
                        startDestination = "pantalla1"
                    ) {
                        composable("pantalla1") { Content01(navController) }
                        composable("pantalla2") { Content02(navController) }
                    }
                }
            )
        }
    }
```

```
39  class MainActivity : ComponentActivity() {
40      override fun onCreate(savedInstanceState: Bundle?) {
41          super.onCreate(savedInstanceState)
42          enableEdgeToEdge()
43          setContent {
44              ViewContainer()
45              }
46          }
47      }
48
49
50  @Preview(showBackground = true)
51  @SuppressLint( ...value = "UnusedMaterial3ScaffoldPaddingParameter")
52  @Composable
53  fun ViewContainer() {
54      val navController = rememberNavController()
55      MaterialTheme {
56          Scaffold(
57              content = {
58                  NavHost(
59                      navController = navController,
60                      startDestination = "pantalla1"
61                  ) {
62                      composable( route = "pantalla1") { Content01(navController) }
63                      composable( route = "pantalla2") { Content02(navController) }
64                  }
65              }
66          )
67      }
68  }
```

2. Ejemplo de menú de navegación con NavController

```
@Composable
    fun Content01(navController: NavController) {

        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {

            Button(
                onClick = { navController.navigate("pantalla2") },
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color.Blue,
                    contentColor = Color.White
                ),
                modifier = Modifier.width(200.dp)
            ) {
                Text("Ir a Pantalla 2")
            }
        }
    }
```

```
70      @Composable
71      fun Content01(navController: NavController) {
72
73          Column(
74              modifier = Modifier.fillMaxSize(),
75              horizontalAlignment = Alignment.CenterHorizontally,
76              verticalArrangement = Arrangement.Center
77          ) {
78
79              Button(
80                  onClick = { navController.navigate( route = "pantalla2") },
81                  colors = ButtonDefaults.buttonColors(
82                      containerColor = Color.Blue,
83                      contentColor = Color.White
84                  ),
85                  modifier = Modifier.width(200.dp)
86              ) {
87                  Text("Ir a Pantalla 2")
88              }
89          }
90      }
```

2. Ejemplo de menú de navegación con NavController

```
@Composable
    fun Content02(navController: NavController) {

        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {

            Text("Estás en Pantalla 2", color = Color.White)

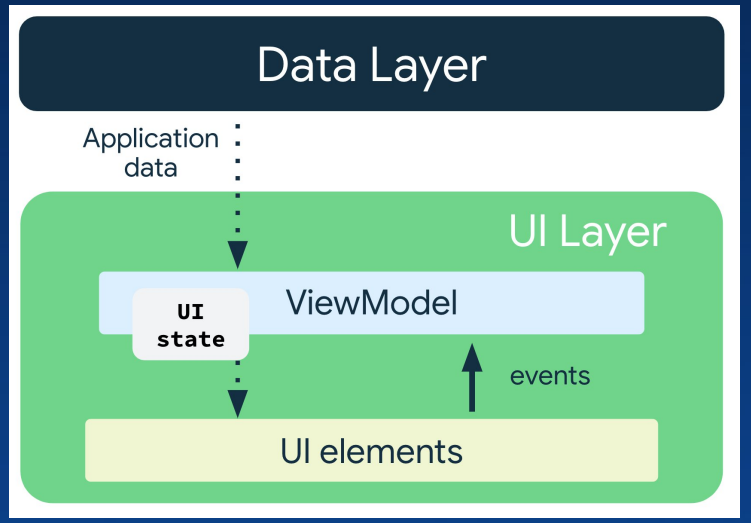
            Spacer(modifier = Modifier.height(20.dp))

            Button(
                onClick = { navController.popBackStack() },
                colors = ButtonDefaults.buttonColors(
                    containerColor = Color.Red,
                    contentColor = Color.White
                ),
                modifier = Modifier.width(200.dp)
            ) {
                Text("Volver")
            }
        }
    }
```

```
92  @Composable
93  fun Content02(navController: NavController) {
94
95      Column(
96          modifier = Modifier.fillMaxSize(),
97          horizontalAlignment = Alignment.CenterHorizontally,
98          verticalArrangement = Arrangement.Center
99      ) {
100
101         Text("Estás en Pantalla 2", color = Color.White)
102
103         Spacer(modifier = Modifier.height(20.dp))
104
105         Button(
106             onClick = { navController.popBackStack() },
107             colors = ButtonDefaults.buttonColors(
108                 containerColor = Color.Red,
109                 contentColor = Color.White
110             ),
111             modifier = Modifier.width(200.dp)
112         ) {
113             Text("Volver")
114         }
115     }
116 }
```

3. NavController y ViewModel

- Cuando usamos NavController y vamos a otro composable, si rotamos el móvil, la pantalla sigue en el mismo composable, es decir, no vuelve al inicial.
- Sin embargo, todos los datos que han ido variando, contadores, listas mutables, etc... Sí que volverían a su estado inicial.
- Para evitarlo usamos la clase ViewModel, donde guardaremos todos los parámetros que queramos que persistan cuando navegamos entre composables.



3. NavController y ViewModel

```
class HistoricalViewModel: ViewModel() {
        val historical = mutableStateListOf<String>()

        var likes by mutableIntStateOf(0)
            private set

        fun register (message: String){
            historical.add(message)
        }

        fun addLike() {
            likes++
        }

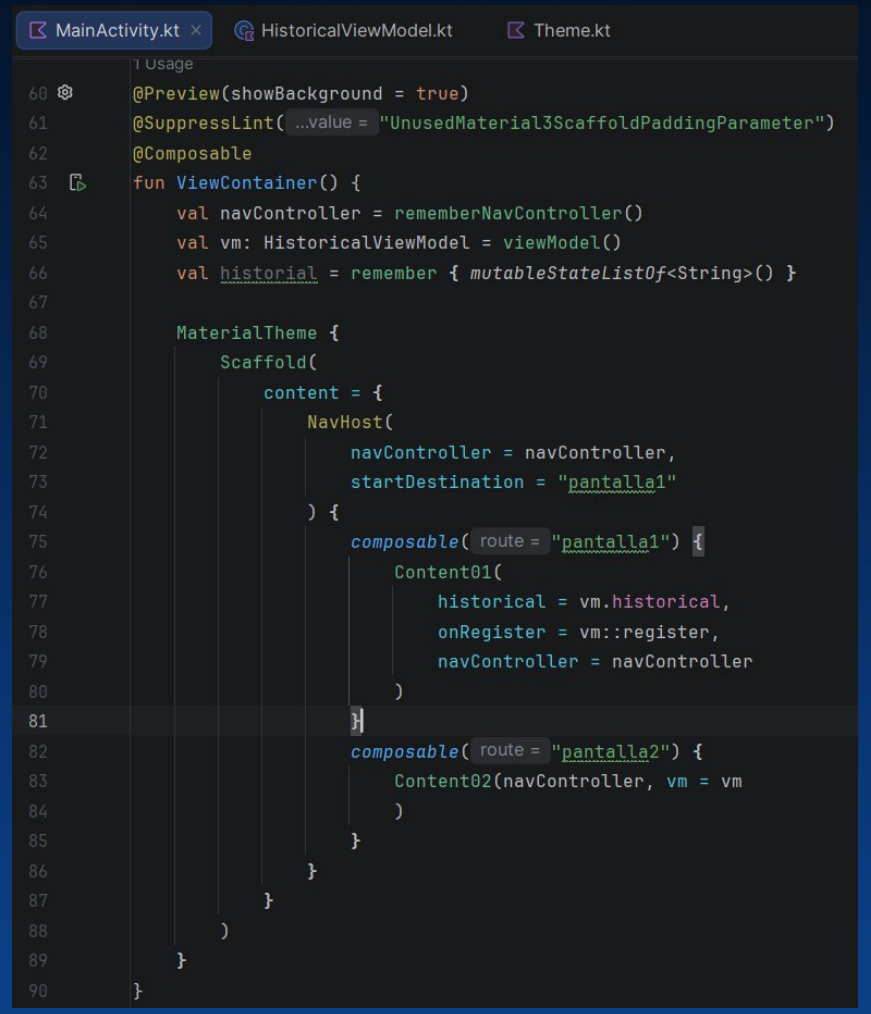
        fun getlikes():Int {
            return likes
        }
    }
```

MainActivity.kt | HistoricalViewModel.kt | Theme.kt

```
1   package com.example.navcontrollerapplication.ui.viewmodel
2
3   import ...
9
    3 Usages
10  class HistoricalViewModel: ViewModel() {
        2 Usages
11      val historical = mutableStateListOf<String>()
12
        2 Usages
13      var likes by mutableIntStateOf( value = 0)
14          private set
15
        1 Usage
16      fun register (message: String){
17          historical.add(message)
18      }
19
        1 Usage
20      fun addLike() {
21          likes++
22      }
23
        1 Usage
24      fun getlikes():Int {
25          return likes
26      }
27  }
```

3. NavController y ViewModel

```
@Preview(showBackground = true)
@SuppressLint("UnusedMaterial3ScaffoldPaddingParameter")
@Composable
fun ViewContainer() {
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()
    val historial = remember { mutableStateListOf<String>() }
    PersonalProfileApplicationTheme {
        Scaffold(
            snackbarHost = { SnackbarHost(snackbarHostState) },
            topBar = { Toolbar() },
            content = {
                Content(
                    historial = historial,
                    onRegister = { mensaje ->
                        historial.add(mensaje)
                    })
            },
            floatingActionButton = {
                FAB(
                    snackbarHostState = snackbarHostState,
                    scope = scope,
                    onRegister = { mensaje ->
                        historial.add(mensaje)
                    }
                )
            },
            floatingActionButtonPosition = FabPosition.End
        )
    }
}
```



3. NavController y ViewModel

```
@Composable
        fun Content02(navController: NavController, vm: HistoricalViewModel) {

            Column(
                modifier = Modifier.fillMaxSize(),
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Center
            ) {


                Text("Estás en Pantalla 2", color = Color.White)
                Spacer(modifier = Modifier.height(20.dp))

                Image(
                    painter = painterResource(id = R.drawable.ic_like),
                    contentDescription = "icono like",
                    colorFilter = ColorFilter.tint(Color.Red),
                    modifier = Modifier.clickable {
                        vm.addLike()
                    }
                )

                Text(
                    text = vm.getlikes().toString(),
                    color = Color.Black,
                    modifier = Modifier.padding(start = 4.dp)
                )

                Button(
                    onClick = { navController.popBackStack() },
                    colors = ButtonDefaults.buttonColors(
                        containerColor = Color.Red,
                        contentColor = Color.White
                    ),
                    modifier = Modifier.width(200.dp)
                ) {
                    Text("Volver")
                }
            }
```

```
121  @Composable
122  fun Content02(navController: NavController, vm: HistoricalViewModel) {
123
124      Column(
125          modifier = Modifier.fillMaxSize(),
126          horizontalAlignment = Alignment.CenterHorizontally,
127          verticalArrangement = Arrangement.Center
128      ) {
129
130
131          Text("Estás en Pantalla 2", color = Color.White)
132          Spacer(modifier = Modifier.height(20.dp))
133
134          Image(
135              painter = painterResource(id = R.drawable.ic_like),
136              contentDescription = "icono like",
137              colorFilter = ColorFilter.tint(Color.Red),
138              modifier = Modifier.clickable {
139                  vm.addLike()
140              }
141          )
142
143          Text(
144              text = vm.getlikes().toString(),
145              color = Color.Black,
146              modifier = Modifier.padding(start = 4.dp)
147          )
148
149          Button(
150              onClick = { navController.popBackStack() },
151              colors = ButtonDefaults.buttonColors(
```

- Ejercicios 1-2